
Session 18 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rathi**

College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

Domain: **AIML (Artificial Intelligence and Machine Learning)**

Github Repo Link :

https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments

Github File Link :

https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session18/Assignment18.ipynb

Dataset Used: – Insurance.csv (Already Provided)

Available on Github File Link.

Q1. Data Loading & Initial Analysis

Output Screenshots:

First 10 Rows:

	model	year	price	transmission	mileage	fuelType	tax	mpg	engineSize
0	Fiesta	2017	12000	Automatic	15944	Petrol	150	57.7	1.0
1	Focus	2018	14000	Manual	9083	Petrol	150	57.7	1.0
2	Focus	2017	13000	Manual	12456	Petrol	150	57.7	1.0
3	Fiesta	2019	17500	Manual	10460	Petrol	145	40.3	1.5
4	Fiesta	2019	16500	Automatic	1482	Petrol	145	48.7	1.0
5	Fiesta	2015	10500	Manual	35432	Petrol	145	47.9	1.6
6	Puma	2019	22500	Manual	2029	Petrol	145	50.4	1.0
7	Fiesta	2017	9000	Manual	13054	Petrol	145	54.3	1.2
8	Kuga	2019	25500	Automatic	6894	Diesel	145	42.2	2.0
9	Focus	2018	10000	Manual	48141	Petrol	145	61.4	1.0

Last 5 Rows:

	model	year	price	transmission	mileage	fuelType	tax	mpg	\
17961	B-MAX	2017	8999	Manual	16700	Petrol	150	47.1	
17962	B-MAX	2014	7499	Manual	40700	Petrol	30	57.7	
17963	Focus	2015	9999	Manual	7010	Diesel	20	67.3	
17964	KA	2018	8299	Manual	5007	Petrol	145	57.7	
17965	Focus	2015	8299	Manual	5007	Petrol	22	57.7	

engineSize

17961	1.4
17962	1.0
17963	1.6
17964	1.2
17965	1.0

Shape of Dataset:

(17966, 9)

Data Types:

model	object
year	int64
price	int64
transmission	object
mileage	int64
fuelType	object
tax	int64
mpg	float64
engineSize	float64
dtype:	object

Program Code:

'''

Q1. (Data Loading & Initial Analysis)

Load the Ford Car Dataset using pandas.

Display the first 10 rows and last 5 rows.

Check the shape (number of rows and columns) and data types of all columns.

Write observations about the dataset structure in comments.

'''

```
import pandas as pd
```

```
# Load the dataset
```

```
df = pd.read_csv("ford_car_dataset.csv")
```

```
# Display first 10 rows
```

```
print("First 10 Rows:")
```

```
print(df.head(10))
```

```
# Display last 5 rows
```

```
print("\nLast 5 Rows:")
```

```
print(df.tail())
```

```
# Display shape
```

```
print("\nShape of Dataset:")
```

```
print(df.shape)
```

```
# Display data types
```

```
print("\nData Types:")
```

```
print(df.dtypes)
```

```
# Observations:
```

```
# 1. The dataset contains information about Ford cars and their features.
```

```
# 2. It consists of both numerical and categorical columns.
```

```
# 3. The shape indicates the total number of records and features.
```

```
# 4. Columns such as model and transmission are categorical, while year, price,  
# mileage, mpg, engineSize, and tax are numerical.
```

```
# 5. The dataset is suitable for data analysis and regression-based machine  
learning.
```

Q2.Missing & Duplicate Values

Output Screenshot:

```
Missing Values in Each Column:
```

```
model      0
```

```
year       0
```

```
price      0
```

```
transmission 0
```

```
mileage    0
```

```
fuelType   0
```

```
tax         0
```

```
mpg         0
```

```
engineSize 0
```

```
dtype: int64
```

```
Number of Duplicate Rows: 154
```

```
Duplicate Rows After Removal:
```

```
0
```

Program Code:

```
'''
```

Q2. (Missing & Duplicate Values)

Perform a detailed check on the dataset:

Find the total number of missing values in each column.

Identify and remove duplicate rows if any.

Write a summary of your findings and how you handled them in comments.

```
'''
```

```
# Check missing values
```

```
print("Missing Values in Each Column:")
```

```
print(df.isnull().sum())
```

```
# Check duplicate rows
```

```
duplicate_count = df.duplicated().sum()
```

```
print("\nNumber of Duplicate Rows:", duplicate_count)
```

```
# Remove duplicate rows (if any)
```

```
df = df.drop_duplicates()
```

```
# Verify duplicates after removal
```

```
print("\nDuplicate Rows After Removal:")
```

```
print(df.duplicated().sum())
```

```
# Observations:
```

```
# 1. Checked all columns for missing values using isnull().sum().
```

```
# 2. Counted duplicate records using duplicated().sum().
```

```
# 3. Removed duplicate rows using drop_duplicates().
```

```
# 4. Verified that no duplicate rows remain in the dataset.
```

```
# 5. Removing duplicates improves data quality for analysis and machine learning.
```

Q3.Statistical Summary

Output Screenshot:

```
Statistical Summary:
      year      price      mileage      tax      mpg \
count 17812.000000 17812.000000 17812.000000 17812.000000 17812.000000
mean  2016.862396 12269.556310 23381.146362 113.315012 57.908696
std    2.052039  4736.285417 19419.011045 62.034603 10.132696
min   1996.000000  495.000000  1.000000  0.000000 20.800000
25%   2016.000000 8999.000000 10000.000000 30.000000 52.300000
50%   2017.000000 11288.000000 18277.000000 145.000000 58.900000
75%   2018.000000 15295.000000 31098.500000 145.000000 65.700000
max   2060.000000 54995.000000 177644.000000 580.000000 201.800000

      engineSize
count 17812.000000
mean   1.350623
std    0.432581
min    0.000000
25%    1.000000
50%    1.200000
75%    1.500000
max    5.000000

Price Statistics
Minimum : 495
Maximum : 54995
Mean    : 12269.556310352571
Median  : 11288.0

Mileage Statistics
Minimum : 1
Maximum : 177644
Mean    : 23381.146362003143
Median  : 18277.0

Year Statistics
Minimum : 1996
Maximum : 2060
Mean    : 2016.8623961374353
Median  : 2017.0
```

Program Code:

'''

Q3. (Statistical Summary)

Generate the statistical summary of all numeric columns using describe(). Identify minimum, maximum, mean, and median values for key columns like price, mileage, and year.

'''

```
# Display statistical summary
print("Statistical Summary:")
print(df.describe())
```

```
# Minimum, Maximum, Mean, and Median for selected columns
columns = ["price", "mileage", "year"]
```

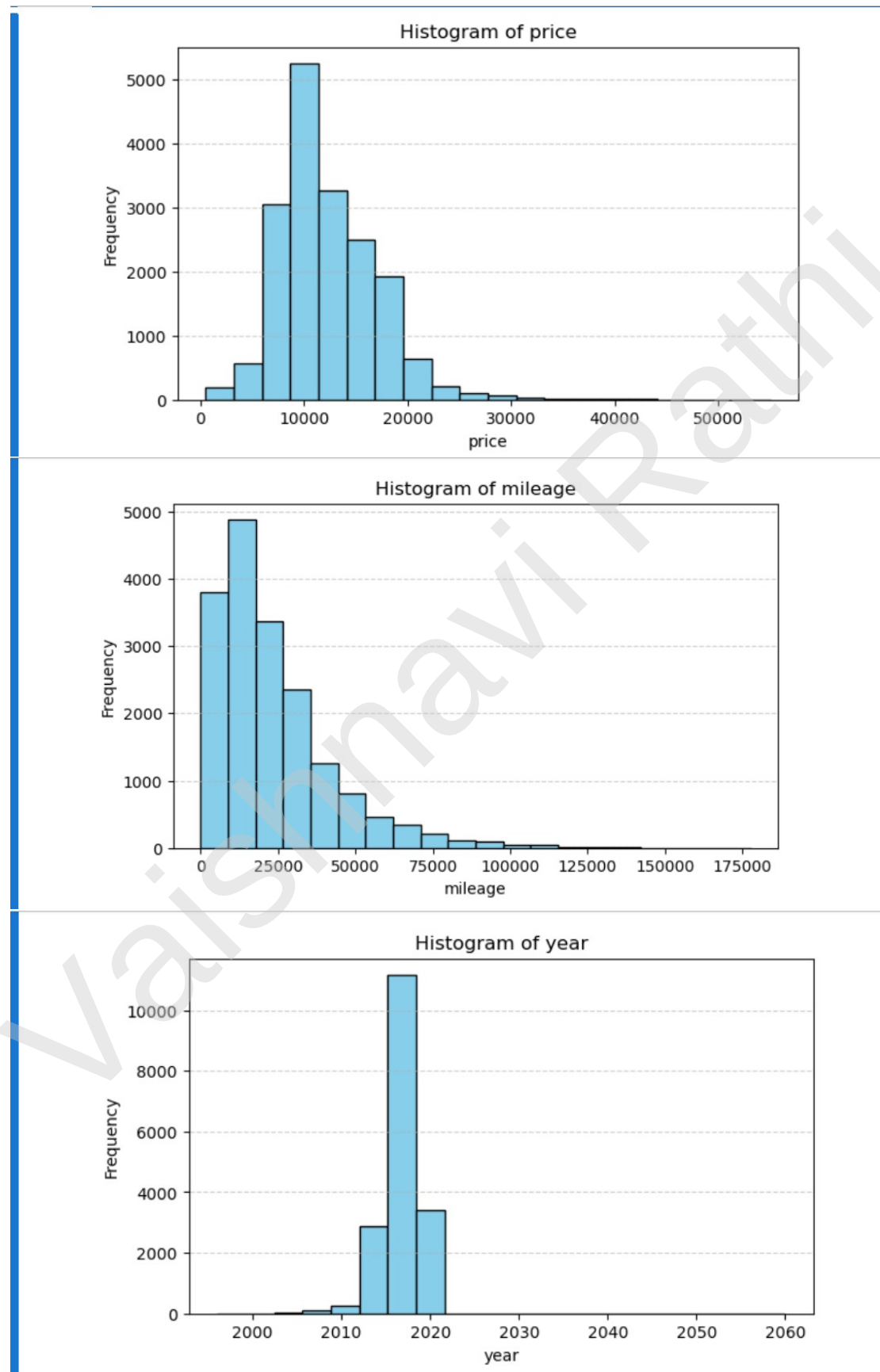
```
for col in columns:
    print(f"\n{col.capitalize()} Statistics")
    print("Minimum :", df[col].min())
    print("Maximum :", df[col].max())
    print("Mean   :", df[col].mean())
    print("Median  :", df[col].median())
```

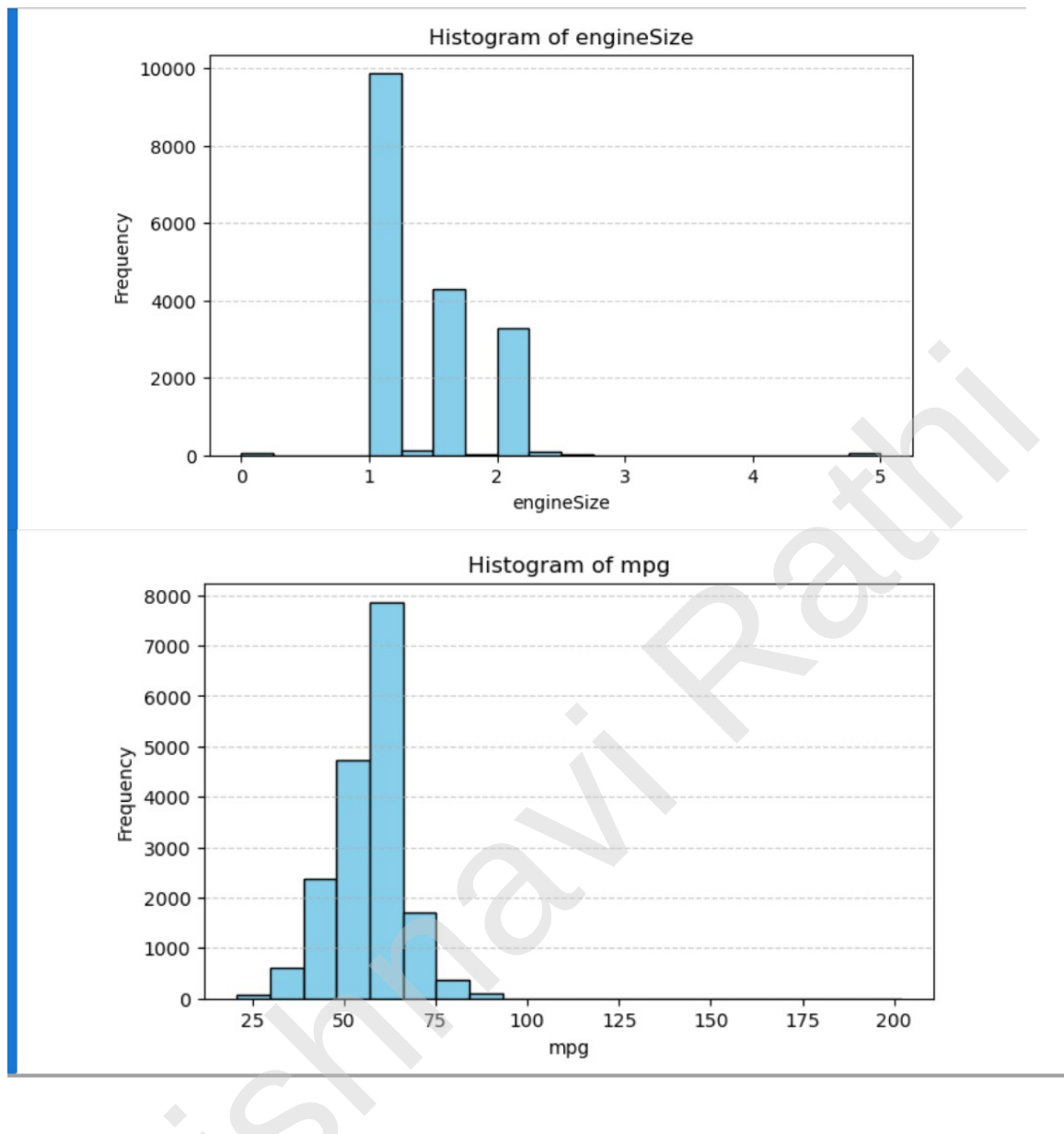
Observations:

- # 1. The statistical summary provides an overview of all numeric columns.
 - # 2. Mean and median indicate the central tendency of the data.
 - # 3. Minimum and maximum values show the range of each feature.
 - # 4. These statistics help in understanding the distribution before further analysis.
-

Q4. Histogram of Numeric Features

Output Screenshot:





Program Code:

```
'''  
Q4. (Histogram of Numeric Features)  
  
Plot histograms for all important numeric columns (price, mileage, year,  
engineSize, mpg).  
Analyze the distribution for each feature and note your findings in comments.  
'''  
  
# Import Library  
import matplotlib.pyplot as plt  
  
# List of numeric columns  
numeric_columns = ["price", "mileage", "year", "engineSize", "mpg"]
```

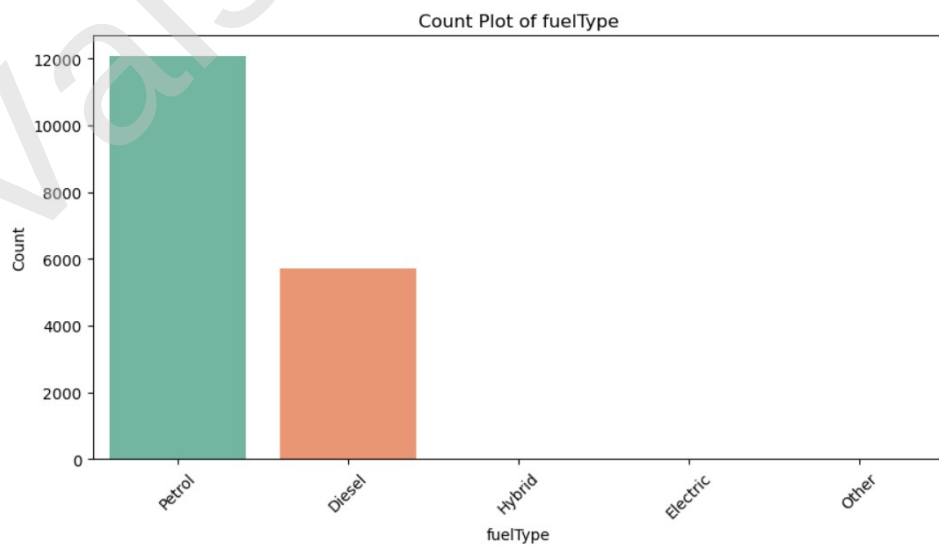
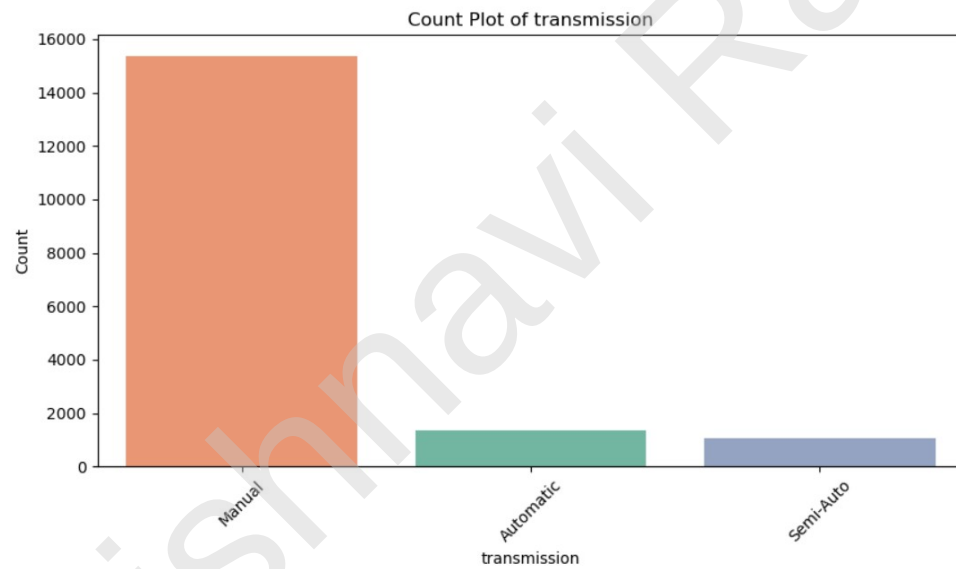
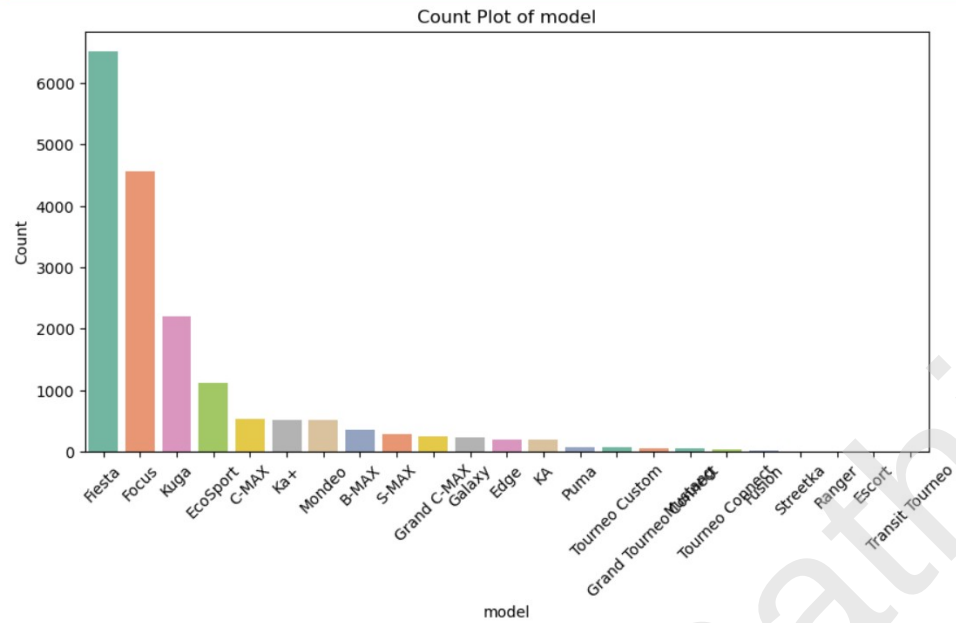
```
# Plot histograms
for col in numeric_columns:
    plt.figure(figsize=(7, 4))
    plt.hist(df[col], bins=20, edgecolor="black", color="skyblue")
    plt.title(f"Histogram of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")
    plt.grid(axis="y", linestyle="--", alpha=0.6)
    plt.show()
```

Observations:

- # 1. Price values are spread over a wide range, with most cars in the mid-price range.
 - # 2. Mileage is right-skewed, indicating that many cars have lower mileage while fewer have very high mileage.
 - # 3. The year distribution shows that most cars are from recent manufacturing years.
 - # 4. Engine size is concentrated around common values such as 1.0L, 1.5L, and 2.0L.
 - # 5. MPG values are mostly clustered within a specific range, with a few extreme values.
-

Q5.Count Plots of Categorical Features

Output Screenshot:



Program Code:

```
'''
Q5. (Count Plots of Categorical Features)

Create count plots for all categorical columns (fuelType, transmission, model, etc.)
using seaborn.
Analyze which categories are most common.

Write insights about market trends based on these plots in comments.
'''

# Import Library
import seaborn as sns

# Select categorical columns
categorical_columns = ["model", "transmission", "fuelType"]
```

```
# Plot count plots
for col in categorical_columns:
    plt.figure(figsize=(10, 5))

    sns.countplot(
        data=df,
        x=col,
        hue=col,
        order=df[col].value_counts().index,
        palette="Set2",
        legend=False
    )

    plt.title(f"Count Plot of {col}")
    plt.xlabel(col)
    plt.ylabel("Count")
    plt.xticks(rotation=45)
    plt.show()
```

Observations:

- # 1. Count plots show the frequency of each category in the dataset.
 - # 2. Some Ford models appear much more frequently than others.
 - # 3. Manual and Automatic are the most common transmission types.
 - # 4. Petrol and Diesel vehicles dominate the dataset, while other fuel types are less common.
 - # 5. These trends indicate higher market availability and customer preference for these categories.
-

Q6. Correlation Heatmap

Output Screenshot:



Program Code:

```
'''
```

Q6. (Correlation Heatmap)

Create a heatmap of the correlation matrix for numeric features.
Use seaborn with proper annotations.

Identify which features are highly correlated with the target variable (price).
Write your observations in comments.
'''

```
# Select only numeric columns
numeric_df = df.select_dtypes(include=["number"])
```

```
# Create correlation matrix
corr_matrix = numeric_df.corr()
```

```
# Plot heatmap
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)
```

```
plt.title("Correlation Heatmap")
plt.show()
```

```
# Display correlation with price
print("Correlation with Price:")
print(corr_matrix["price"].sort_values(ascending=False))
```

Observations:

- # 1. The heatmap shows the correlation between all numeric features.
 - # 2. Features with higher positive correlation are more closely related to price.
 - # 3. Mileage generally has a negative correlation with price, indicating that
cars with higher mileage tend to have lower prices.
 - # 4. Year and engineSize usually show a positive correlation with price,
meaning newer cars and larger engines often have higher prices.
 - # 5. Correlation analysis helps identify important features for regression models.
-

Q7.Feature Identification

Output Screenshot:

Independent Features (Input Features):
['model', 'year', 'transmission', 'mileage', 'fuelType', 'tax', 'mpg', 'engineSize']

Dependent Feature (Target Feature):
price

Program Code:

'''

Q7. (Feature Identification)

Clearly identify and list:

Independent Features (Input Features)

Dependent Feature (Output/Target Feature)

Justify your choice with reasoning.

'''

Independent (Input) Features

X = df.drop("price", axis=1)

Dependent (Target) Feature

y = df["price"]

print("Independent Features (Input Features):")

print(X.columns.tolist())

print("\nDependent Feature (Target Feature):")

print(y.name)

Observations:

1. The independent features are all columns except 'price'.

2. The dependent feature is 'price' because it is the value we want to predict.

3. Features such as year, mileage, transmission, fuelType, mpg, and engineSize

influence the selling price of a Ford car.

4. Therefore, 'price' is selected as the target variable for building a

regression model.

Q8.Encoding Categorical Variables

Output Screenshot:

Before One Hot Encoding:

```
model transmission
0 Fiesta Automatic
1 Focus Manual
2 Focus Manual
3 Fiesta Manual
4 Fiesta Automatic
```

After One Hot Encoding (Sample):

```
model_C-MAX model_EcoSport model_Edge model_Escort model_Fiesta \
0 False False False False True
1 False False False False False
2 False False False False False
3 False False False False True
4 False False False False True
```

```
model_Focus model_Fusion model_Galaxy model_Grand C-MAX \
0 False False False False False
1 True False False False False
2 True False False False False
3 False False False False False
4 False False False False False
```

```
model_Grand Tourneo Connect ... model_Mustang model_Puma model_Ranger \
0 False ... False False False False
1 False ... False False False False
2 False ... False False False False
3 False ... False False False False
4 False ... False False False False
```

```
model_S-MAX model_Streetka model_Tourneo Connect model_Tourneo Custom \
0 False False False False False
1 False False False False False
2 False False False False False
3 False False False False False
4 False False False False False
```

```
model_Transit Tourneo transmission_Manual transmission_Semi-Auto
0 False False False False
1 False False True False
2 False False True False
3 False False True False
4 False False False False
```

[5 rows x 24 columns]

Shape Before Encoding: (17812, 9)

Shape After Encoding: (17812, 34)

Program Code:

```
'''
```

Q8. (Encoding Categorical Variables)

Convert all categorical columns into numeric form using appropriate encoding techniques:

Use One Hot Encoding.

Show before and after transformation for at least 2 columns.

```
'''
```

```
# Display original categorical columns
```

```
print("Before One Hot Encoding:")
```

```
print(df[["model", "transmission"]].head())
```

```
# Apply One Hot Encoding
```

```
encoded_df = pd.get_dummies(
```

```
    df,
```

```
    columns=["model", "transmission", "fuelType"],
```

```
    drop_first=True
```

```
)
```

```
# Display encoded columns
```

```
print("\nAfter One Hot Encoding (Sample):")
```

```
print(encoded_df.filter(regex="model_|transmission_").head())
```

```
print("\nShape Before Encoding:", df.shape)
```

```
print("Shape After Encoding:", encoded_df.shape)
```

```
# Observations:
```

```
# 1. One Hot Encoding converts categorical values into binary (0/1) columns.
```

```
# 2. The original categorical columns are replaced with new encoded columns.
```

```
# 3. The number of columns increases after encoding because each category
```

```
#    becomes a separate feature.
```

```
# 4. This transformation makes the dataset suitable for machine learning models.
```

Q9.Feature Scaling

Output Screenshot:

First 5 Rows of Scaled Data:

	year	mileage	tax	mpg	engineSize
0	0.067059	-0.382994	0.591380	-0.020597	-0.810561
1	0.554393	-0.736317	0.591380	-0.020597	-0.810561
2	0.067059	-0.562616	0.591380	-0.020597	-0.810561
3	1.041726	-0.665405	0.510777	-1.737858	0.345325
4	1.041726	-1.127749	0.510777	-0.908836	-0.810561

Program Code:

'''

Q9. (Feature Scaling)

Apply Standard Scaling (using StandardScaler from sklearn) on the numeric

independent features.

Show the first 5 rows of scaled data.

'''

```
from sklearn.preprocessing import StandardScaler
```

```
# Select numeric independent features
```

```
numeric_features = ["year", "mileage", "tax", "mpg", "engineSize"]
```

```
# Create StandardScaler object
```

```
scaler = StandardScaler()
```

```
# Apply feature scaling
```

```
scaled_data = scaler.fit_transform(df[numeric_features])
```

```
# Convert scaled data into a DataFrame
```

```
scaled_df = pd.DataFrame(scaled_data, columns=numeric_features)
```

```
# Display first 5 rows
```

```
print("First 5 Rows of Scaled Data:")
```

```
print(scaled_df.head())
```

```
# Observations:
```

```
# 1. Standard Scaling transforms the data so that each feature has
```

```
# a mean close to 0 and a standard deviation close to 1.
```

```
# 2. Scaling helps improve the performance of many machine learning  
algorithms.
```

```
# 3. Only numeric independent features are scaled, while categorical features
```

```
# are handled separately using encoding.
```

Q10. Mini Project - Complete Preprocessing Pipeline

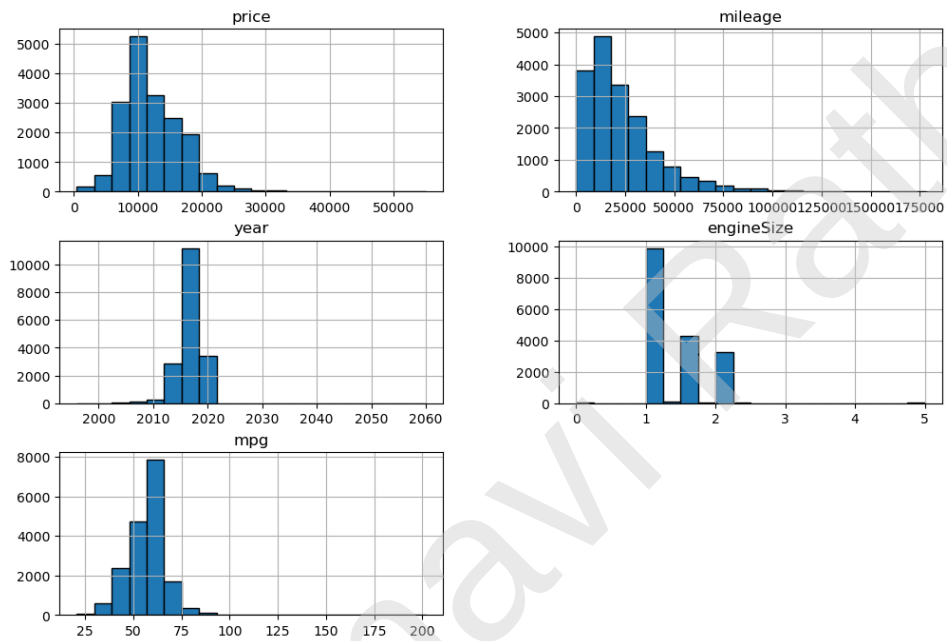
Output Screenshot:

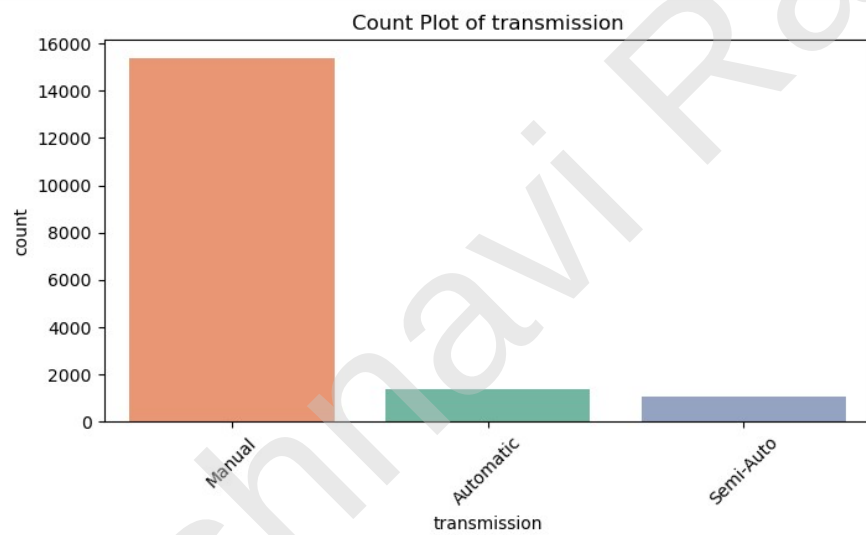
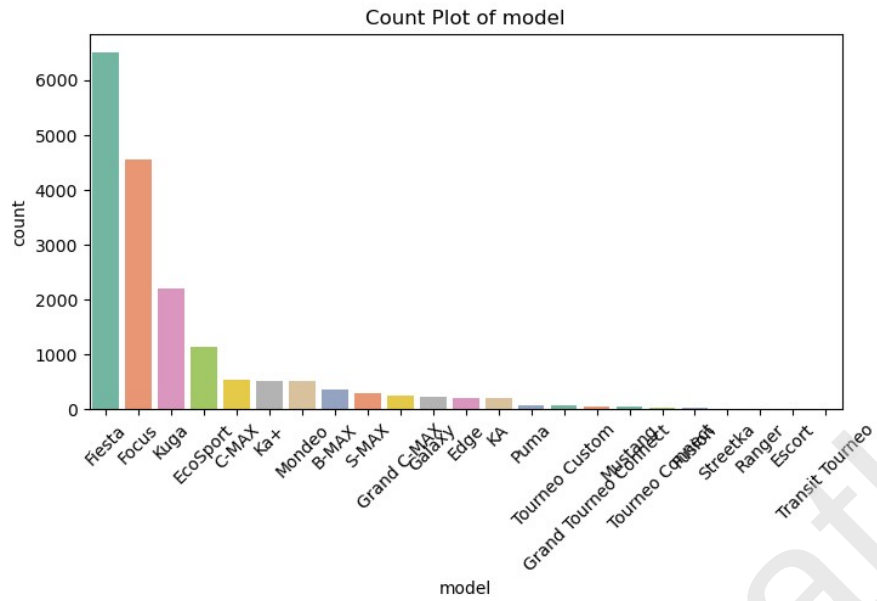
Missing Values:

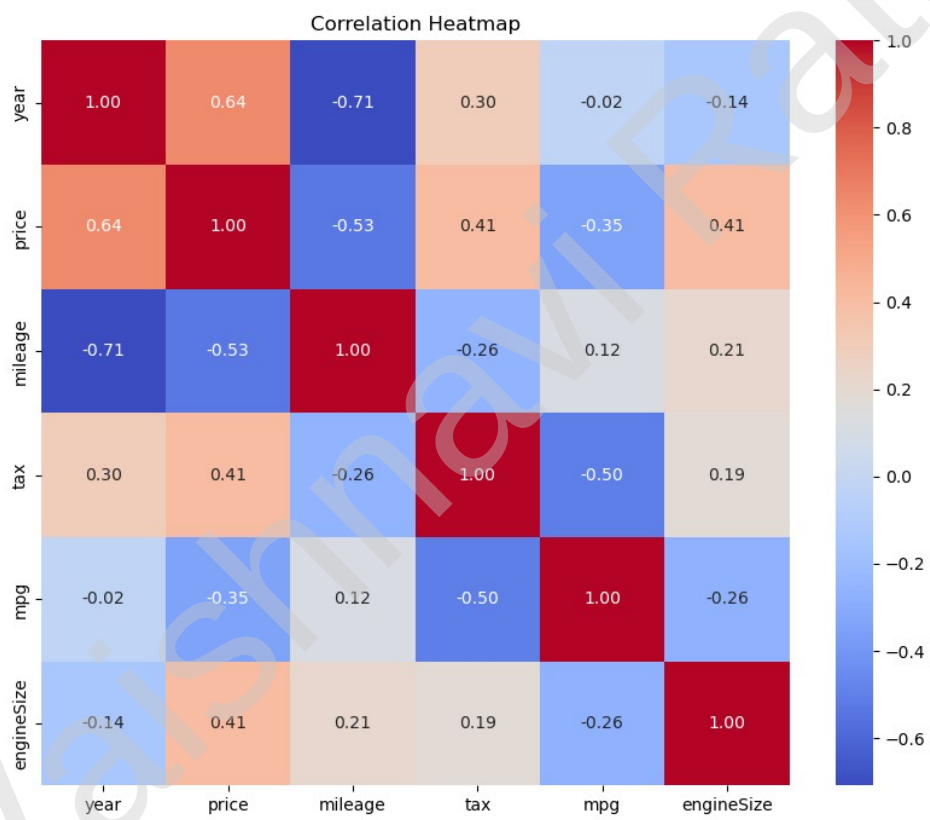
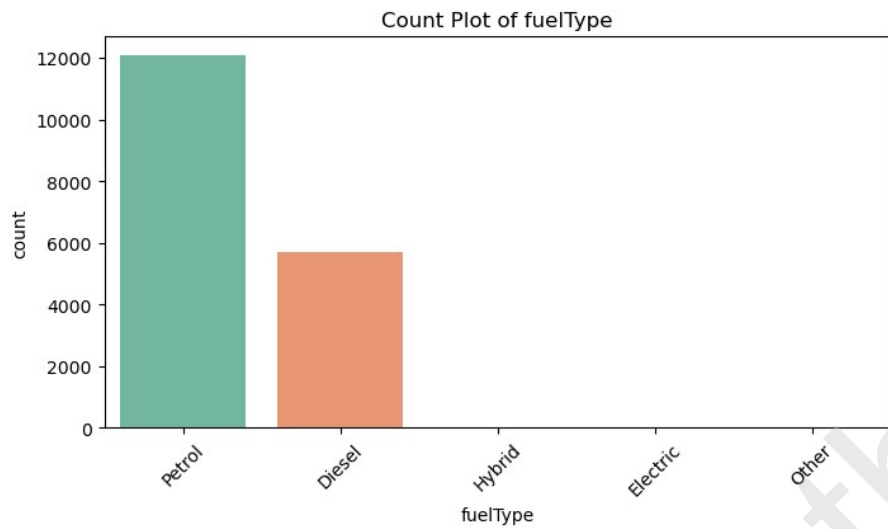
```
model      0
year       0
price      0
transmission 0
mileage    0
fuelType   0
tax        0
mpg        0
engineSize 0
dtype: int64
```

Duplicate Rows After Removal: 0

Histograms of Numeric Features







Input Features:
['model', 'year', 'transmission', 'mileage', 'fuelType', 'tax', 'mpg', 'engineSize']

Output Feature:
price

First 5 Rows of Preprocessed Data:

	year	mileage	tax	mpg	engineSize	model_C-MAX	\
0	0.067059	-0.382994	0.591380	-0.020597	-0.810561	False	
1	0.554393	-0.736317	0.591380	-0.020597	-0.810561	False	
2	0.067059	-0.562616	0.591380	-0.020597	-0.810561	False	
3	1.041726	-0.665405	0.510777	-1.737858	0.345325	False	
4	1.041726	-1.127749	0.510777	-0.908836	-0.810561	False	

	model_EcoSport	model_Edge	model_Escort	model_Fiesta	...	\
0	False	False	False	True	...	
1	False	False	False	False	...	
2	False	False	False	False	...	
3	False	False	False	True	...	
4	False	False	False	True	...	

	model_Streetka	model_Tourneo Connect	model_Tourneo Custom	\
0	False	False	False	
1	False	False	False	
2	False	False	False	
3	False	False	False	
4	False	False	False	

	model_Transit Tourneo	transmission_Manual	transmission_Semi-Auto	\
0	False	False	False	
1	False	True	False	
2	False	True	False	
3	False	True	False	
4	False	False	False	

	fuelType_Electric	fuelType_Hybrid	fuelType_Other	fuelType_Petrol
0	False	False	False	True
1	False	False	False	True
2	False	False	False	True
3	False	False	False	True
4	False	False	False	True

[5 rows x 33 columns]

Program Code:

```
'''
```

Q10. (Mini Project - Complete Preprocessing Pipeline)

Create a complete preprocessing pipeline for the Ford Car Dataset:

1. Load and clean the data (handle missing & duplicate values).
2. Perform EDA (histograms + count plots + heatmap).
3. Identify input and output features.
4. Encode categorical variables.
5. Scale the numeric features.

```
'''
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Step 1: Data Cleaning
print("Missing Values:")
print(df.isnull().sum())
```

```
df = df.drop_duplicates()
print("\nDuplicate Rows After Removal:", df.duplicated().sum())
```

```
# Step 2: Exploratory Data Analysis (EDA)
```

```
# Histogram
df[["price", "mileage", "year", "engineSize", "mpg"]].hist(
    figsize=(12, 8),
    bins=20,
    edgecolor="black"
)
plt.suptitle("Histograms of Numeric Features")
plt.show()
```

```
# Count Plots
categorical_columns = ["model", "transmission", "fuelType"]
```

```
for col in categorical_columns:
    plt.figure(figsize=(8, 4))
    sns.countplot(
        data=df,
        x=col,
        hue=col,
        palette="Set2",
        legend=False,
        order=df[col].value_counts().index
    )
    plt.title(f"Count Plot of {col}")
    plt.xticks(rotation=45)
```

```

plt.show()

# Correlation Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(
    df.select_dtypes(include="number").corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
plt.title("Correlation Heatmap")
plt.show()

# Step 3: Identify Features
X = df.drop("price", axis=1)
y = df["price"]

print("\nInput Features:")
print(X.columns.tolist())

print("\nOutput Feature:")
print(y.name)

# Step 4: One Hot Encoding
X_encoded = pd.get_dummies(
    X,
    columns=["model", "transmission", "fuelType"],
    drop_first=True
)

# Step 5: Feature Scaling
numeric_columns = ["year", "mileage", "tax", "mpg", "engineSize"]

scaler = StandardScaler()

X_encoded[numeric_columns] = scaler.fit_transform(
    X_encoded[numeric_columns]
)

print("\nFirst 5 Rows of Preprocessed Data:")
print(X_encoded.head())

```

Summary:

- # 1. Loaded and cleaned the dataset by checking missing values and removing duplicates.**
 - # 2. Performed EDA using histograms, count plots, and a correlation heatmap.**
 - # 3. Selected 'price' as the target variable and the remaining columns as input features.**
 - # 4. Applied One Hot Encoding to convert categorical features into numeric form.**
 - # 5. Applied Standard Scaling to the numeric features.**
 - # 6. The dataset is now ready for training a machine learning regression model.**
-